

# **SE2012 -Software Engineering**

**Theory :2 Hrs/Week**

**Tutorial : 1 Hrs/Week**

**Credit: 3**

# **Unit I: Introduction To Software Engineering (07 hrs )**

# **Unit -1**

## **Introduction to Software Engineering**

### **Syllabus**

**Introduction to Software Engineering, Software crisis, evolution of software engineering, characteristics of good software, attributes of software quality, software engineering layers, software process framework, software development life cycle (SDLC), generic process model, Waterfall model, Incremental model, Prototype model, Spiral model, V-model, Agile overview, Scrum roles, Case Study: Online Shopping System.**

# Definition of Software

- Software is a collection of programs, procedures, data, and related documentation that instructs a computer system on how to perform specific tasks or solve particular problems.

In software engineering, software includes:

- Programs (instructions)
- Data structures (data used by programs)
- Documentation (user manuals, design documents)

## Characteristics of Software

- **Software is developed, not manufactured**

It is created through design and coding, not produced in a factory like hardware.

- **Software does not wear out**

Unlike hardware, software does not physically degrade; it may fail due to errors or changes.

- **Software is intangible**

It cannot be physically touched; it exists as programs and data.

- **Software is complex**

It involves many components, logic, and interactions.

- **Software is custom-built**

Most software is developed to meet specific user requirements.

- **Software is easy to reproduce**

Copying software has negligible cost compared to development.

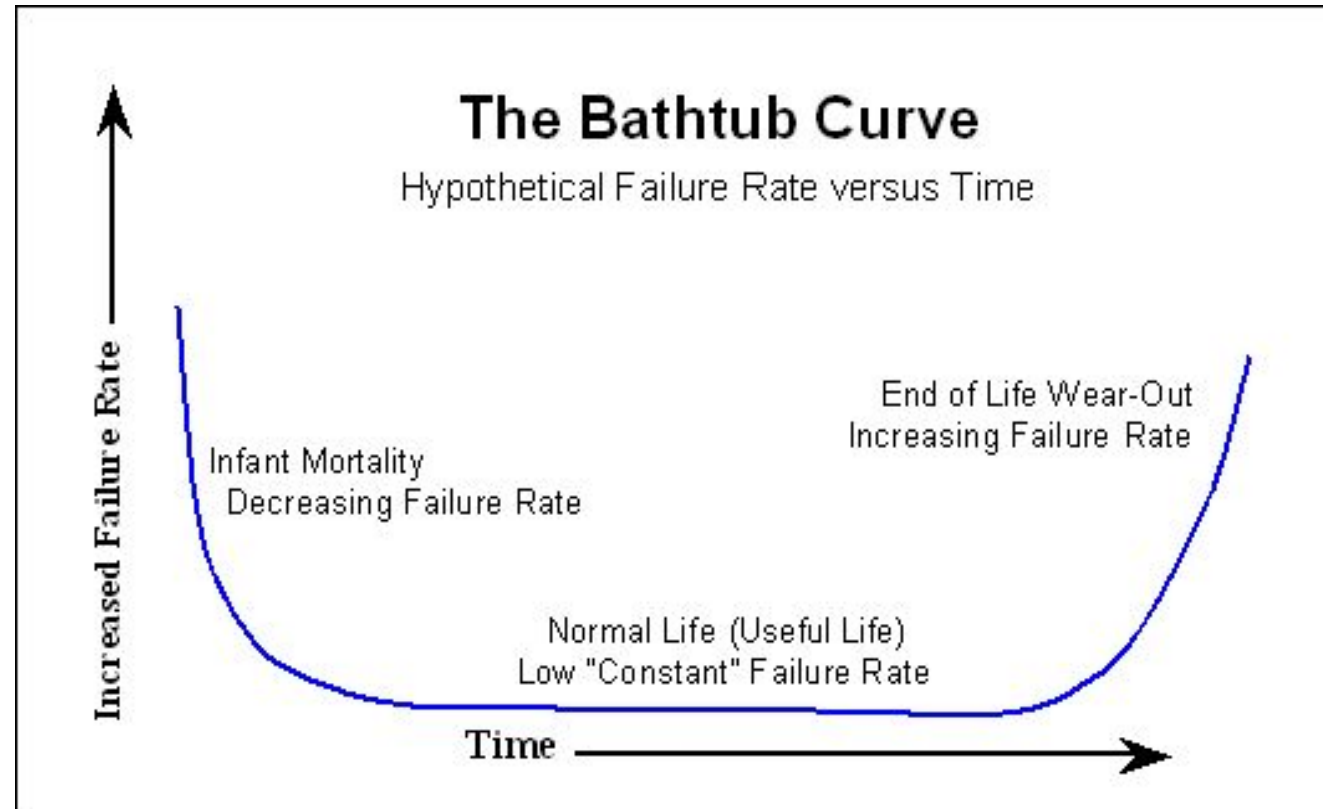
- **Software requires maintenance**

Changes are needed to fix bugs, improve performance, or adapt to new environments.

- **Software evolves over time**

It is continuously modified to meet changing user needs.

# Bathtub Curve for Hardware failure



The curve has three distinct phases, shaped like a bathtub:

### **1. Infant Mortality Phase (Early Life)**

- **High failure rate**
- Caused by manufacturing defects, poor assembly, or installation errors
- Failures decrease rapidly over time
- Examples: faulty chips, loose connections

### **2. Useful Life Phase (Normal Operation)**

- **Low and constant failure rate**
- Random failures occur
- Hardware operates as expected
- This is the **longest and most reliable period**



### **3. Wear-Out Phase**

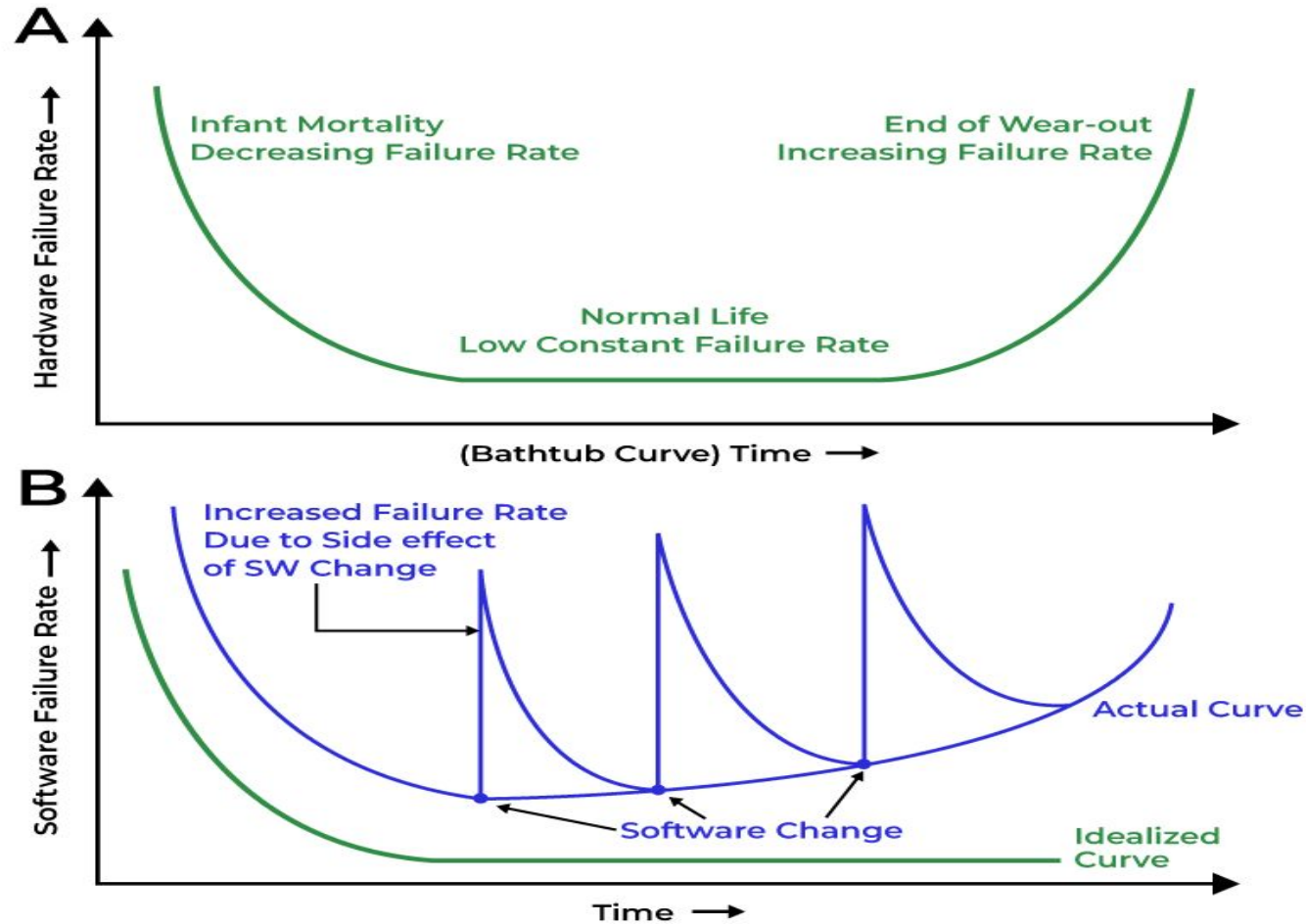
- **Increasing failure rate**

- Caused by aging, material fatigue, corrosion, and wear
- Examples: disk wear, capacitor degradation

### **Importance in Software Engineering**

- Helps in **maintenance planning**
- Assists in **hardware replacement decisions**
- Supports **reliability and risk analysis**

# Bathtub Curve for Hardware/Software failure



In **software engineering**, the **Bathtub Curve** represents the **failure rate of software over time**, but its behavior is **different from hardware** because software does **not physically wear out**.

## **Phases of Software Bathtub Curve**

### **1. Early Life (Debugging Phase)**

- **High failure rate**
- Caused by design errors, coding bugs, and integration issues
- Failures decrease as bugs are detected and fixed

### **2. Useful Life (Stable Operation)**

- **Low and nearly constant failure rate**
- Software operates reliably
- Few failures if no changes are made

### 3. Change / Maintenance Phase

- Failure rate **increases again**
- Caused by:
  - New feature additions
  - Modifications and patches
  - Environment changes
- Introducing changes may create **new defects**

### Key Difference from Hardware

- Software **does not wear out**
- Failure increase is due to **changes**, not aging

# **Types of Software (Software Engineering)**

## **1. System Software**

- Manages and controls computer hardware
- Provides a platform for application software

**Examples:** Operating Systems (Windows, Linux), device drivers, compilers

## **2. Application Software**

- Designed to perform user-specific tasks

**Examples:** Word processors, web browsers, banking software, hospital management systems

## **3. Utility Software**

- Helps in system maintenance and optimization

**Examples:** Antivirus, disk cleanup, backup tools

#### **4. Embedded Software**

- Built into hardware devices to control their functions

**Examples:** Washing machines, cars, microwave ovens

#### **5. Real-Time Software**

- Responds to inputs within strict time constraints

**Examples:** Air traffic control systems, medical monitoring systems

#### **6. Web-Based Software**

- Runs on web servers and accessed through browsers

**Examples:** E-commerce websites, web portals, cloud applications

#### **7. Mobile Software**

- Developed for smartphones and tablets

**Examples:** Android and iOS applications

# **Software Crisis (Software Engineering)**

**Software Crisis** refers to the **difficulties faced in developing, maintaining, and managing large and complex software systems**, especially during the late 1960s and early 1970s.

## **Causes of Software Crisis**

1. Increasing **size and complexity** of software
2. **Poor project management**
3. Lack of **standard development methodologies**
4. Inaccurate **cost and time estimation**
5. Insufficient **testing and documentation**
6. Rapid **changes in user requirements**

## Effects of Software Crisis

- Projects delivered **late**
- **High development cost**
- Software often **unreliable**
- Difficult and expensive **maintenance**
- Failure to meet user requirements

## Solutions to Software Crisis

- Introduction of **Software Engineering** discipline
- Use of **structured programming**
- Adoption of **SDLC models**
- Improved **project management tools**
- Emphasis on **testing, documentation, and quality assurance**



# Evolution of Software Engineering

The **evolution of software engineering** describes how software development progressed from ad-hoc programming to a disciplined engineering approach.

## 1. Early Programming Era (1950s–1960s)

- Programs were **small and simple**
- Written mainly in **machine and assembly languages**
- No formal methods or documentation
- Focus was only on making the program work

## 2. Software Crisis Era (Late 1960s–1970s)

- Rapid growth in **software size and complexity**
- Projects failed due to **cost overruns, delays, and poor quality**
- Term “**Software Engineering**” was introduced (1968 NATO Conference)
- Need for systematic development methods identified

### **3. Structured Programming Era (1970s–1980s)**

- Introduction of **high-level languages** (C, Pascal)
- Use of **structured design and modular programming**
- Emphasis on documentation and testing
- Early **SDLC models** like Waterfall

### **4. Object-Oriented Era (1980s–1990s)**

- Adoption of **Object-Oriented Programming (OOP)**
- Concepts like classes, objects, inheritance
- UML introduced for modeling
- Improved reuse and maintainability

## 5. Component-Based & Reuse-Oriented Era (1990s–2000s)

- Focus on **reusable components**
- Use of libraries, frameworks, and APIs
- Faster development and reduced cost

## 6. Agile and Iterative Era (2000s–Present)

- Agile methods (Scrum, XP)
- Emphasis on **customer collaboration and flexibility**
- Continuous integration and delivery
- Rapid response to changing requirements

## 7. Modern Software Engineering (Present)

- DevOps, Cloud Computing, Microservices
- AI-driven development and automation
- Emphasis on scalability, security, and reliability

# **Characteristics of Good Software (Software Engineering)**

## **1. Correctness**

Software performs all required functions accurately as per specifications.

## **2. Reliability**

Software operates without failure for a specified period under given conditions.

## **3. Efficiency**

Uses system resources (CPU, memory, time) optimally.

## **4. Usability**

Easy to learn, understand, and use by end users.

## **5. Maintainability**

Easy to modify, fix bugs, and enhance.

## **Portability**

- Can be used across different platforms and environments.

## **Security**

- Protects data and system resources from unauthorized access.

## **Scalability**

- Can handle growth in users, data, or workload.

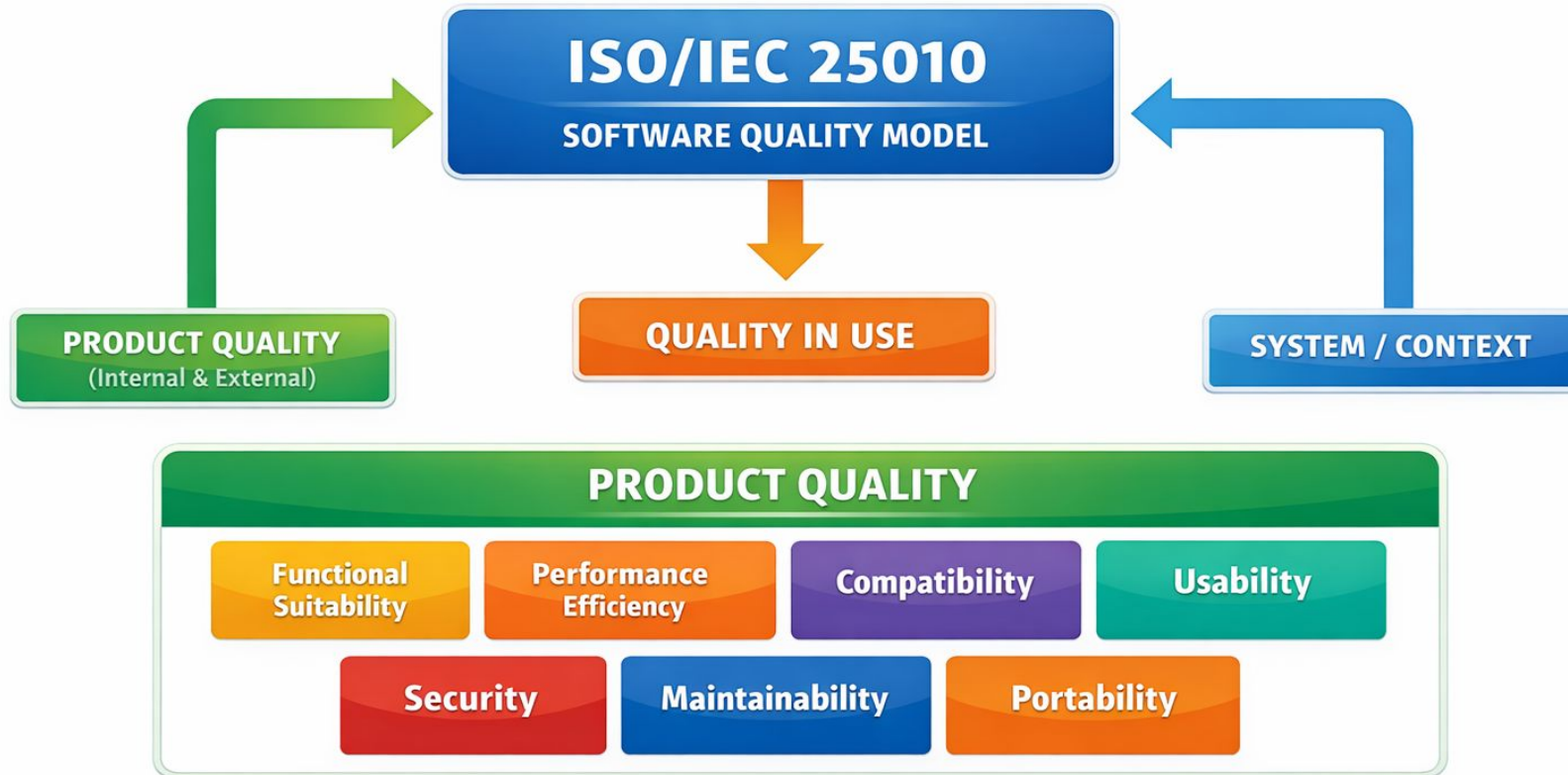
## **Reusability**

- Components can be reused in other software systems.

## **Robustness**

- Handles errors and unexpected inputs gracefully.

# Attributes of software quality



**Software Quality Attributes** (also called *quality characteristics*) describe how good a software system is beyond just correct functionality. Key attributes include:

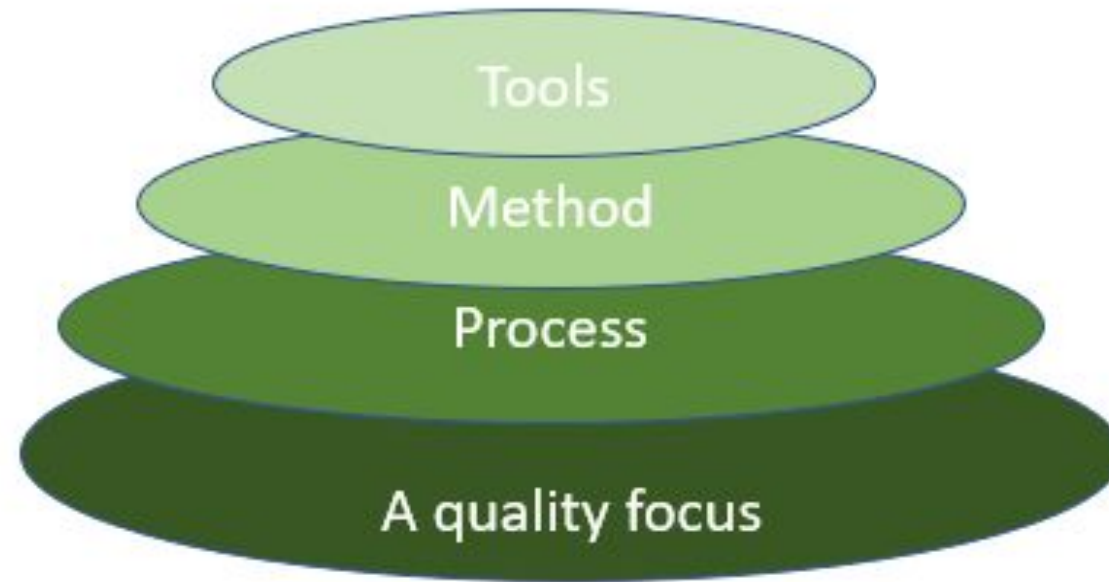
- 1.Functionality** – The degree to which the software meets stated and implied requirements correctly.
- 2.Reliability** – Ability of the software to perform consistently without failure under specified conditions.
- 3.Usability** – Ease of use, learning, and user satisfaction while interacting with the software.
- 4.Efficiency (Performance)** – Optimal use of system resources such as time, memory, and processing power.
- 5.Maintainability** – Ease with which the software can be modified to fix bugs, improve performance, or adapt to changes.

- **Portability** – Ability of the software to run on different platforms or environments with minimal changes.
- **Security** – Protection of data and resources against unauthorized access and attacks.
- **Scalability** – Ability to handle increased workload without performance degradation.
- **Reusability** – Degree to which software components can be reused in other applications.
- **Testability** – Ease of testing the software to ensure it works as intended.



# Software Engineering Layers

- ❑ Software engineering is a fully layered technology, to develop software we need to go from one layer to another. All the layers are connected and each layer demands the fulfillment of the previous layer.



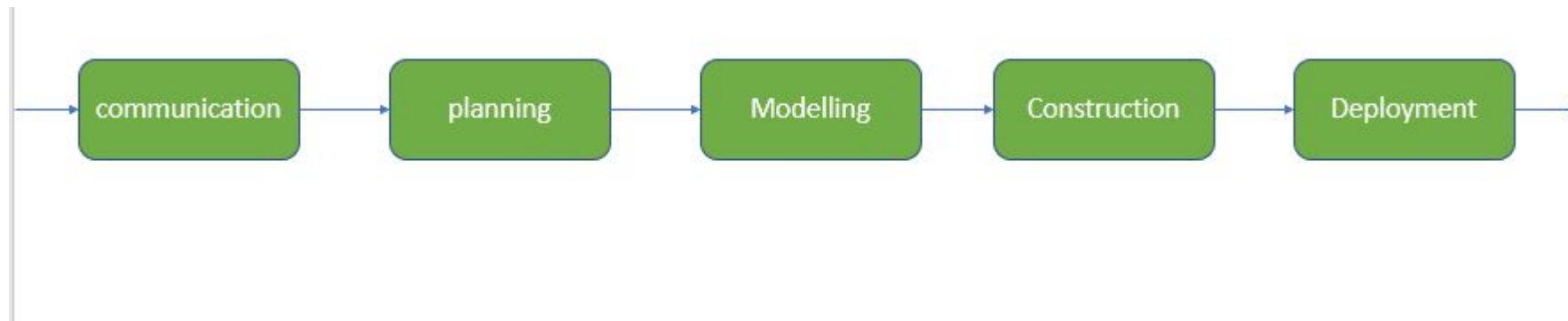
*The diagram shows the layers of software development*

## Layered technology is divided into four parts:

**1. A quality focus:** It defines the continuous process improvement principles of software. It provides integrity that means providing security to the software so that data can be accessed by only an authorized person, no outsider can access the data. It also focuses on maintainability and usability.

- Emphasizes quality assurance, standards, and continuous improvement.
- Includes reviews, audits, and metrics.

**Process:** It is the foundation or base layer of software engineering. It is key that binds all the layers together which enables the development of software before the deadline or on time. Process defines a framework that must be established for the effective delivery of software engineering technology. The software process covers all the activities, actions, and tasks required to be carried out for software development.



## Process activities are listed below:-

- **Communication:** It is the first and foremost thing for the development of software. Communication is necessary to know the actual demand of the client.
- **Planning:** It basically means drawing a map for reduced the complication of development.
- **Modeling:** In this process, a model is created according to the client for better understanding.
- **Construction:** It includes the coding and testing of the problem.
- **Deployment:-** It includes the delivery of software to the client for evaluation and feedback.
- Defines the framework for software development (SDLC).
- Activities: communication, planning, modeling, construction, deployment.

**3. Method:** During the process of software development the answers to all "how-to-do" questions are given by method. It has the information of all the tasks which includes communication, requirement analysis, design modeling, program construction, testing, and support.

- Provides technical “how-to” methods.
- Includes requirements analysis, design models, coding techniques, and testing strategies.

**4. Tools:** Software engineering tools provide a self-operating system for processes and methods. Tools are integrated which means information created by one tool can be used by another.

- Automated or semi-automated tools supporting methods and process.
- Examples: IDEs, version control, CASE tools, testing tools.

# Software Process Framework - Software Engineering

- ❑ A **Software Process Framework** is a structured approach that defines the steps, tasks, and activities involved in software development. This framework serves as a **foundation for software engineering**, guiding the development team through various stages to ensure a **systematic and efficient process**.
- ❑ A Software Process Framework helps in project planning, risk management, and quality assurance by detailing the chronological order of actions.
- ❑ It includes task sets, umbrella activities, and process framework activities, all essential for a **successful software development lifecycle**. Utilizing a well-defined Software Process Framework enhances productivity, consistency, and the overall quality of the software product

# What is a Software Process Framework?

Software Process Framework details the steps and chronological order of a process. Since it serves as a foundation for them, it is utilized in most applications. Task sets, umbrella activities, and process framework activities all define the characteristics of the software development process. Software Process includes:

- 1.Tasks:** They focus on a small, specific objective.
- 2.Action:** It is a set of tasks that produce a major work product.
- 3.Activities:** Activities are groups of related tasks and actions for a major objective.

## What Is a Software Development Framework?

A software development framework is a structured set of tools, libraries, best practices, and guidelines that help developers build software applications. Think of it as a template or foundation that provides the basic structure and components needed for a software project.

## Software Process

### Process Frameworks

#### Umbrella Activities

Action #1.l  
Task sets

|            |
|------------|
| Work Task  |
| Milestone  |
| SQA Points |

Action #1.k  
Task sets

|            |
|------------|
| Work Task  |
| Milestone  |
| SQA Points |

Action #1.n  
Task sets

|            |
|------------|
| Work Task  |
| Milestone  |
| SQA Points |



*Software Process Framework*

## Key Points

- 1.Foundation:** It gives a basic structure or template for developing software, so developers don't have to start from scratch.
- 2.Components and Tools:** It includes pre-built components and tools that make development faster and easier.
- 3.Best Practices and Guidelines:** It offers best practices and guidelines to ensure the software is built in an organized and efficient way.
- 4.Customization:** Developers can modify and add new functions to customize the framework to their specific needs.

## Advantages of Software Development Framework

A **Software Development Framework** offers numerous benefits that streamline the **software development process** and enhance the quality and efficiency of the final product. Here are some key advantages:

- 1.Increased Productivity:** Frameworks provide pre-built components and tools, allowing developers to focus on specific application logic rather than reinventing the wheel.
- 2.Consistent Quality:** By following best practices and standardized processes, frameworks help ensure consistent code quality and structure across the project.
- 3.Reduced Development Time:** With ready-to-use templates and libraries, developers can significantly cut down on the time needed to build applications from scratch.



- 4.**Better Maintainability:** A structured framework makes the codebase more organized and easier to understand, which simplifies maintenance and updates.
- 5.**Enhanced Security:** Frameworks often include built-in security features and follow industry best practices, reducing the risk of vulnerabilities.
- 6.**Scalability:** Frameworks are designed to handle growth, making it easier to scale applications as user demand increases.

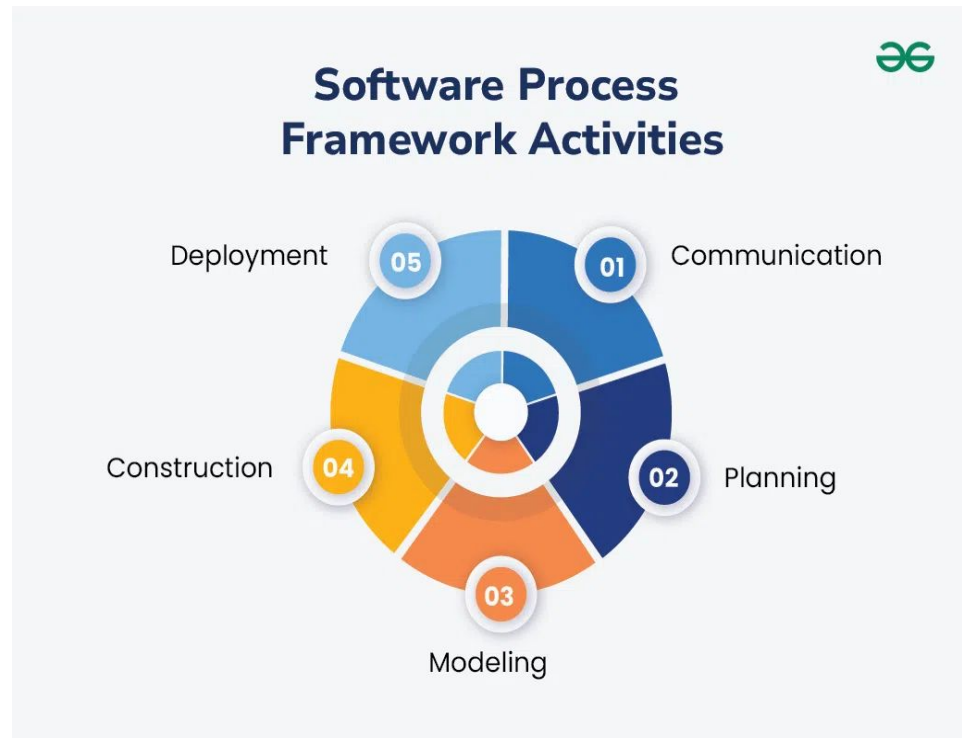
### **Dis-advantages of Software Development Framework**

While **Software Development Frameworks** offer several advantages, they also come with certain drawbacks that developers and organizations should consider:

- 1. **Learning Curve:** Frameworks often have a steep learning curve, requiring developers to invest time and effort in understanding the framework's architecture, conventions, and best practices.
- 2. **Restrictions:** Some frameworks impose constraints and limitations on how developers can design and implement certain features, potentially limiting flexibility and creativity.
- 3. **Complexity Overhead:** In some cases, frameworks introduce unnecessary complexity, especially for smaller or simpler projects, which can lead to over-engineering.

- 4.Performance Overhead:** Using a framework may introduce additional layers of abstraction and overhead, which can impact the performance of the application, particularly in resource-intensive environments.
- 5.Vendor Lock-in:** Depending heavily on a specific framework can lead to vendor lock-in, making it challenging to switch to alternative technologies or frameworks in the future.

## Software Process Framework Activities



*Software Process Framework Activities*

# Software Development Life Cycle (SDLC)

- ❑ The Software Development Life Cycle (SDLC) is a structured framework used by software organizations to design, develop, and test high-quality software. It defines the entire process of software production, from the initial idea to the final deployment and maintenance.
- ❑ The goal of the SDLC is to produce high-quality software that meets or exceeds customer expectations, reaches completion within time and cost estimates, and is efficient to maintain.



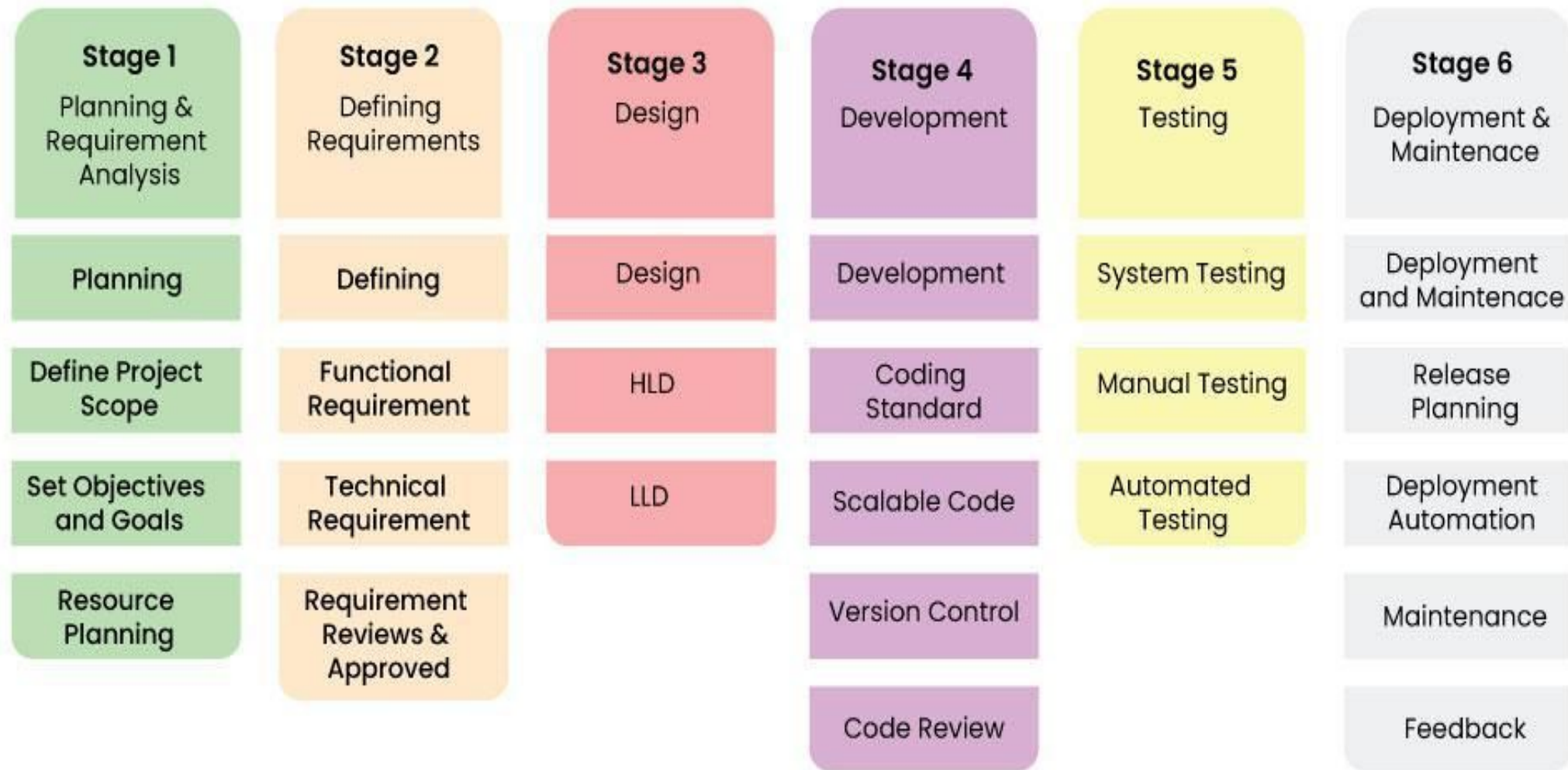
## Why is SDLC Important?

Without a structured SDLC, software development becomes chaotic. Teams may miss requirements, blow budgets, or deliver buggy code that doesn't solve the user's problem. SDLC provides:

- **Visibility:** Stakeholders know exactly what is happening at every stage.
- **Quality Control:** Testing is integrated into the process, not an afterthought.
- **Risk Management:** Potential pitfalls are identified during the planning phase.
- **Cost Estimation:** Helps in predicting timelines and budgets accurately.

## Stages of the Software Development Life Cycle

SDLC specifies the tasks to be performed at various stages by a software engineer or developer. It ensures that the end product is able to meet the customer's expectations and fits within the overall budget. Hence, it's vital for a software developer to have prior knowledge of this software development process. SDLC is a collection of these six stages, and the stages of SDLC are as follows:



## 6 Stages of Software Development Life Cycle



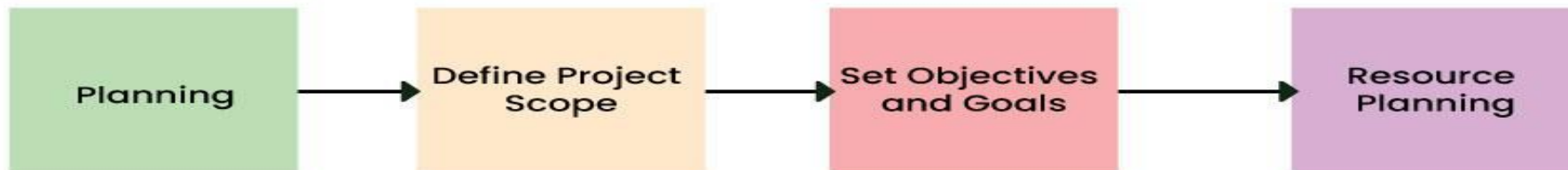
The SDLC Model involves six phases or stages while developing any software.

## Stage 1: Planning and Requirement Analysis

This is the most fundamental stage. Before writing code, the team must define what they are building and why.

- **Activities:** Feasibility studies (technical, operational, economic), resource allocation, project scheduling, and cost estimation.
- **Output:** Project Plan, Feasibility Report.
- **Key Players:** Senior Engineers, Project Managers, Stakeholders.

### Stage-1: Planning and Requirement Analysis

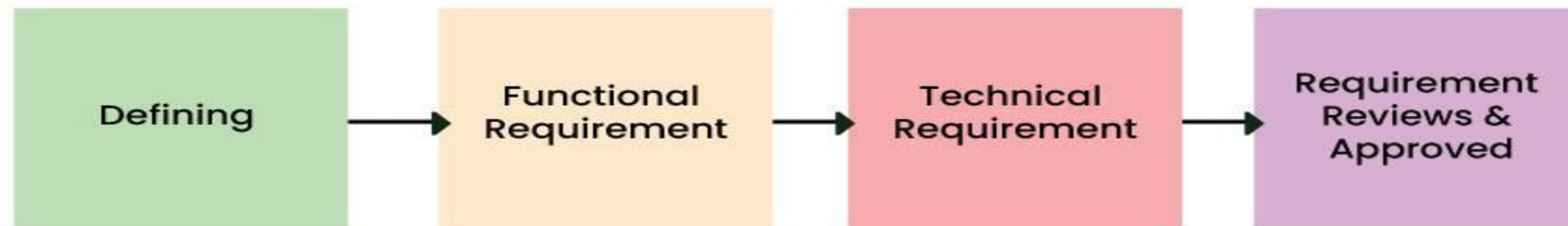


## Stage 2: Defining Requirements (SRS)

Once the plan is approved, the specific product requirements must be defined and documented. This is done through the **Software Requirement Specification (SRS)** document.

- **Activities:** Gathering detailed functional and non-functional requirements from customers.
- **Output:** SRS Document (The "Bible" for the development team).
- **Key Players:** Business Analysts (BAs), Product Owners

### Stage-2: Defining Requirements



## Stage 3: Designing Architecture

The requirements are turned into a technical blueprint. This phase defines the overall system architecture and technology stack.

- **High-Level Design (HLD):** Defines the architecture, database design, and relationships between modules.
- **Low-Level Design (LLD):** Defines the logic of individual components, API interfaces, and database tables.
- **Output:** Design Document Specification (DDS).
- **Key Players:** System Architects, Lead Developers.

### Stage-3: Designing Architecture



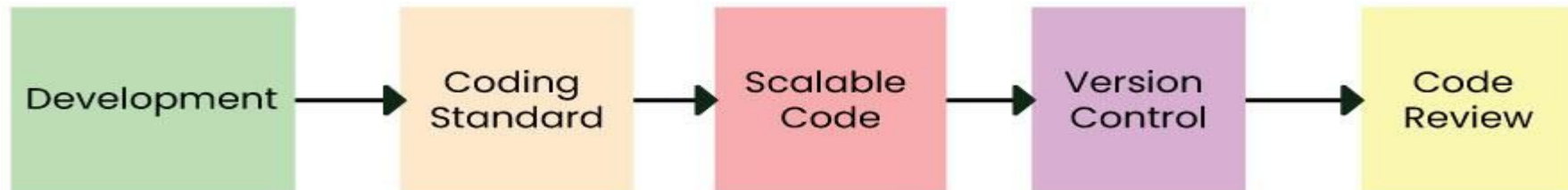


## Stage 4: Development (Coding)

This is the longest phase where the actual building takes place. Developers write code based on the Design Document.

- **Activities:** Coding, Code Reviews, Unit Testing, and Static Code Analysis.
- **Tools:** Compilers, Debuggers, IDEs (VS Code, IntelliJ), Version Control (Git).
- **Output:** Source Code, Executable Software.
- **Key Players:** Developers (Frontend, Backend, Full Stack).

### Stage-4: Developing Product



## Stage 5: Testing

Once the code is written, it moves to the Quality Assurance (QA) team. The goal is to find bugs before the customer does.

### Types of Testing:

- **Unit Testing:** Testing individual functions.
- **Integration Testing:** Ensuring modules work together.
- **System Testing:** Testing the entire application flow.
- **User Acceptance Testing (UAT):** Verifying the software meets business needs.

**Output:** Bug Reports, Test Cases, Quality Report.

**Key Players:** QA Engineers, Testers.

Stage-5: Product Testing and Integration

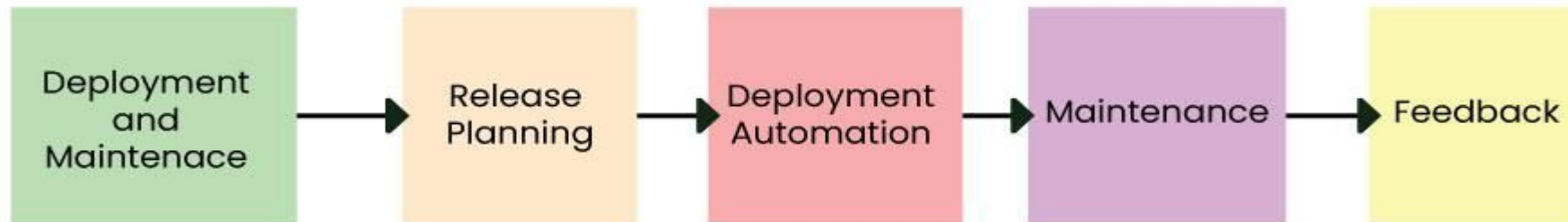


## Stage 6: Deployment

The software is released to the end-users. In modern DevOps environments, this is often automated via CI/CD pipelines.

- **Activities:** Setting up production environments, deploying code, and smoke testing the live environment.
- **Output:** Live Application.
- **Key Players:** DevOps Engineers, Release Managers.

### Stage 6: Deployment and Maintenance of Products



## Stage 7: Maintenance

The cycle doesn't end at deployment. Software must be maintained to remain useful.

- **Activities:** Bug fixing, upgrading libraries, performance tuning, and adding new features.
- **Output:** Patches, Updates, New Versions.
- **Key Players:** Support Engineers, Developers.

## Software Development Life Cycle Models

Software Development Models are structured frameworks that guide the planning, execution, and delivery of software projects. They define the **sequence of development stages**, such as requirements, design, coding, testing, and deployment.

# Generic process model

## Overview

The generic process model is an abstraction of the software development process. It is used in most software since it provides a base for them.

The generic process model encompasses the following five steps:

- ☐ Communication
- ☐ Planning
- ☐ Modelling
- ☐ Construction
- ☐ Deployment

## Communication

In this step, we communicate with the clients and end-users.

- We discuss the requirements of the project with the users.
- The users give suggestions on the project. If any changes are difficult to implement, we work on alternative ideas.

## **Planning**

- In this step, we plan the steps for project development. After completing the final discussion, we report on the project.
- Planning plays a key role in the software development process.
- We discuss the risks involved in the project.

## **Modelling**

- In this step, we create a model to understand the project in the real world. We showcase the model to all the developers. If changes are required, we implement them in this step.
- We develop a practical model to get a better understanding of the project.

## **Construction**

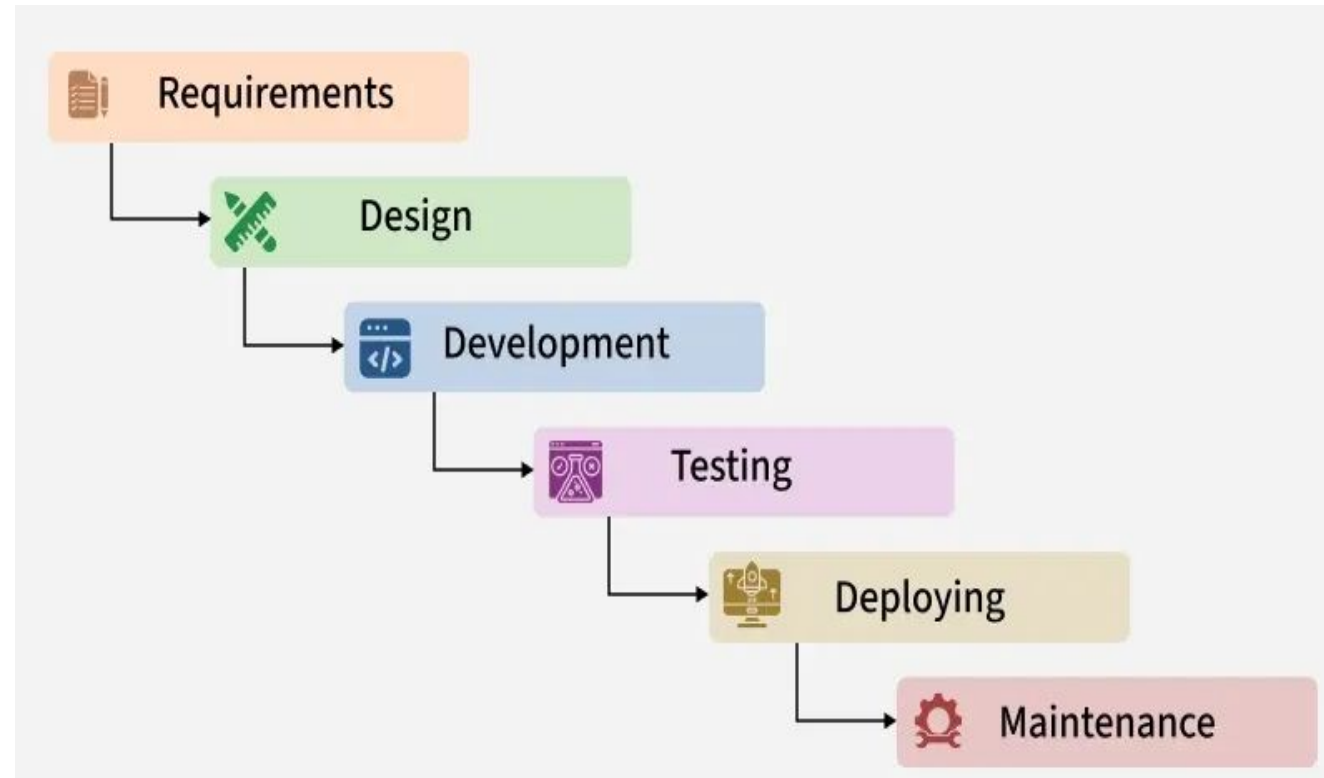
- In this step, we follow a procedure to develop the final product.
- If any code is required for the project development, we implement it in this phase.
- We also test the project in this phase.

## **Deployment**

- In this phase, we submit the project to the clients for their feedback and add any missing requirements.
- We get the client feedback.
- Depending on the feedback form, we make the appropriate changes.

# Waterfall Model

- ❑ The Waterfall Model is a traditional software development methodology that follows a linear, phase-by-phase approach, where each phase must be completed before moving to the next.
- ❑ It does not allow backtracking and permits only minimal changes once a phase is completed. The model is simple, well-structured, and easy to manage, offering high predictability and clearly defined milestones throughout the development process.





## **Features of the Waterfall Model**

- **Sequential Approach**

Development follows a linear, step-by-step process where each phase is completed before moving to the next.

- **Document-Driven Process**

Detailed documentation is created at every stage to clearly define requirements, design, and progress.

- **Quality Control**

Strong emphasis is placed on verification and testing to ensure each phase meets its defined requirements.

- **Detailed Planning**

The project scope, schedule, and deliverables are carefully planned and monitored throughout the lifecycle.

## Phases of Waterfall Model

Classical Waterfall Model divides the life cycle into a set of phases. The development process can be considered as a sequential flow in the waterfall Model. The different sequential phases of the classical waterfall model are as follow:

### 1. Requirements Analysis and Specification

This phase focuses on clearly understanding and documenting the customer's needs.

- **Requirement Gathering and Analysis:**

Customer requirements are collected and carefully examined to remove errors, confusion, and inconsistencies.

- **Requirement Specification:**

The approved requirements are documented in the Software Requirement Specification (SRS), which serves as a formal agreement between the customer and the development team.

## 2. Design

In this phase, the requirements from the SRS are converted into a system design that can be implemented in code.

- **High-Level Design (HLD):**

Defines the overall system architecture, major components, and their interactions.

- **Low-Level Design (LLD):**

Provides detailed design of each component, including logic and data flow, to guide developers during coding.

- All designs are documented in the Software Design Document (SDD).

### 3. Development

This phase involves converting the design into actual working software.

- Developers write source code based on the design documents.
- Suitable programming languages and tools are used.
- Unit testing is performed to verify that each module works correctly on its own.

## 4. Testing and Deployment

This phase ensures that the integrated software functions correctly and is successfully delivered for real world use.

**Testing:** After unit testing, software modules are integrated incrementally and tested at each stage. Once all modules are successfully integrated, full system testing is performed to ensure the system functions as expected.

- Alpha Testing: Performed by the development team.
- Beta Testing: Performed by selected end users.
- Acceptance Testing: Performed by the customer to approve the software.

**Deployment:** After successful testing, the software is deployed to a live environment for end users. This phase includes environment setup, user training, and final checks to ensure smooth operation in real world conditions.

## 5. Maintenance

Maintenance ensures the software continues to function effectively after deployment.

- **Corrective Maintenance:** Fixes errors found after release.
- **Perfective Maintenance:** Enhances features based on user needs.
- **Adaptive Maintenance:** Updates software to work in new environments, such as new hardware or operating systems.

## Advantages

- **Easy to Understand:**

The model is straightforward and simple, making it easy to learn and follow.

- **Sequential Processing:**

Each phase is completed one at a time, ensuring an organized development flow.

- **Well-Defined Stages:**

Every phase has clear objectives and deliverables, reducing confusion.

- **Clear Milestones:**

Progress is easy to track due to clearly defined milestones.

- **Strong Documentation:**

All processes, actions, and results are thoroughly documented.

- **Encourages Best Practices:**

Promotes disciplined practices such as define before design and design before *coding*.

- **Suitable for Small and Stable Projects:**

Works well for smaller projects where requirements are clearly understood and unlikely to change.

## When to Use Waterfall Model?

- **Well-Defined Requirements:**

Requirements are clear, stable, and fully documented before development begins.

- **Minimal Changes Expected:**

The project scope is unlikely to change during development.

- **Small to Medium-Sized Projects:**

Suitable for projects with limited complexity and a clear development path.

- **Predictable and Low Risk:**

Risks are known, manageable, and can be addressed early in the development cycle.

- **Strict Regulatory Compliance:**

Ideal when detailed documentation and adherence to standards or regulations are mandatory.

- **Client Prefers a Sequential Approach:**

Best when the client wants a step-by-step, linear development process.

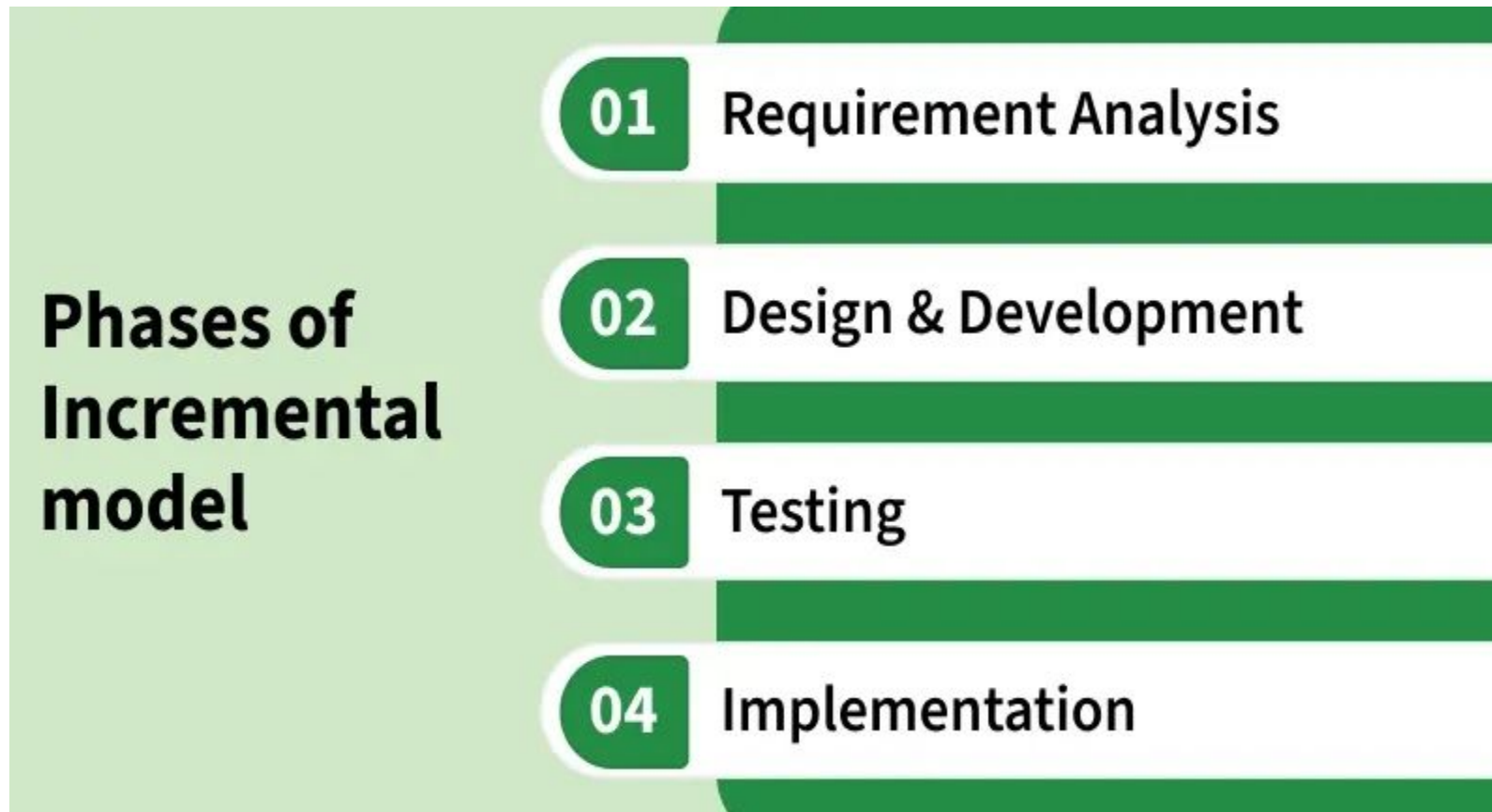
- **Limited Resources:**

Works well when resources are limited and need careful planning and allocation.



# Incremental Process Model

- ❑ The Incremental Process Model is a software development approach in which the system is developed and delivered in small, manageable parts called increments.
- ❑ Each increment adds new functionality to the existing system until the complete product is built.



# Incremental Process Model

Software is not delivered all at once. Instead, development is divided into multiple increments, and each increment goes through requirements, design, development, testing, and implementation phases. Every new release improves the previous version.

- **Step-by-Step Development:** The system is built gradually in small parts.
- **Early Working Software:** A usable version is delivered early.
- **Customer Feedback:** Feedback after each increment improves future releases.
- **Flexible to Changes:** New features can be added in later increments.
- **Structured yet Iterative:** Combines planning of Waterfall with flexibility of iteration.

## **Phases of Incremental Model**

### **1. Requirement Analysis**

Requirements for the current increment are gathered and clearly defined.

### **2. Design & Development**

The system design is prepared, and new features are developed.

### **3. Testing**

Each increment is tested to ensure new and existing features work correctly.

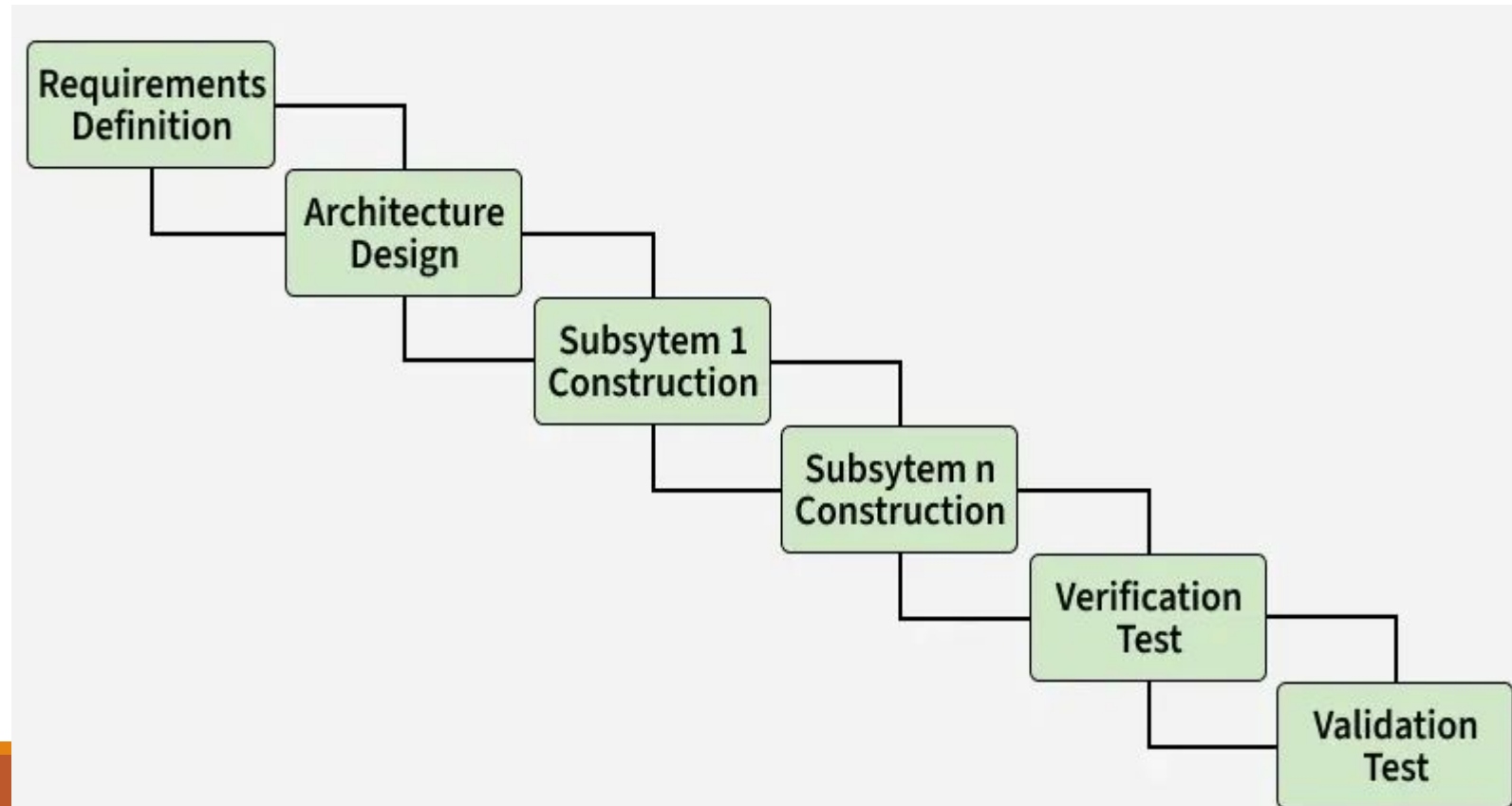
### **4. Implementation**

The tested increment is delivered and integrated with previous versions.

These steps repeat until the final system is completed.

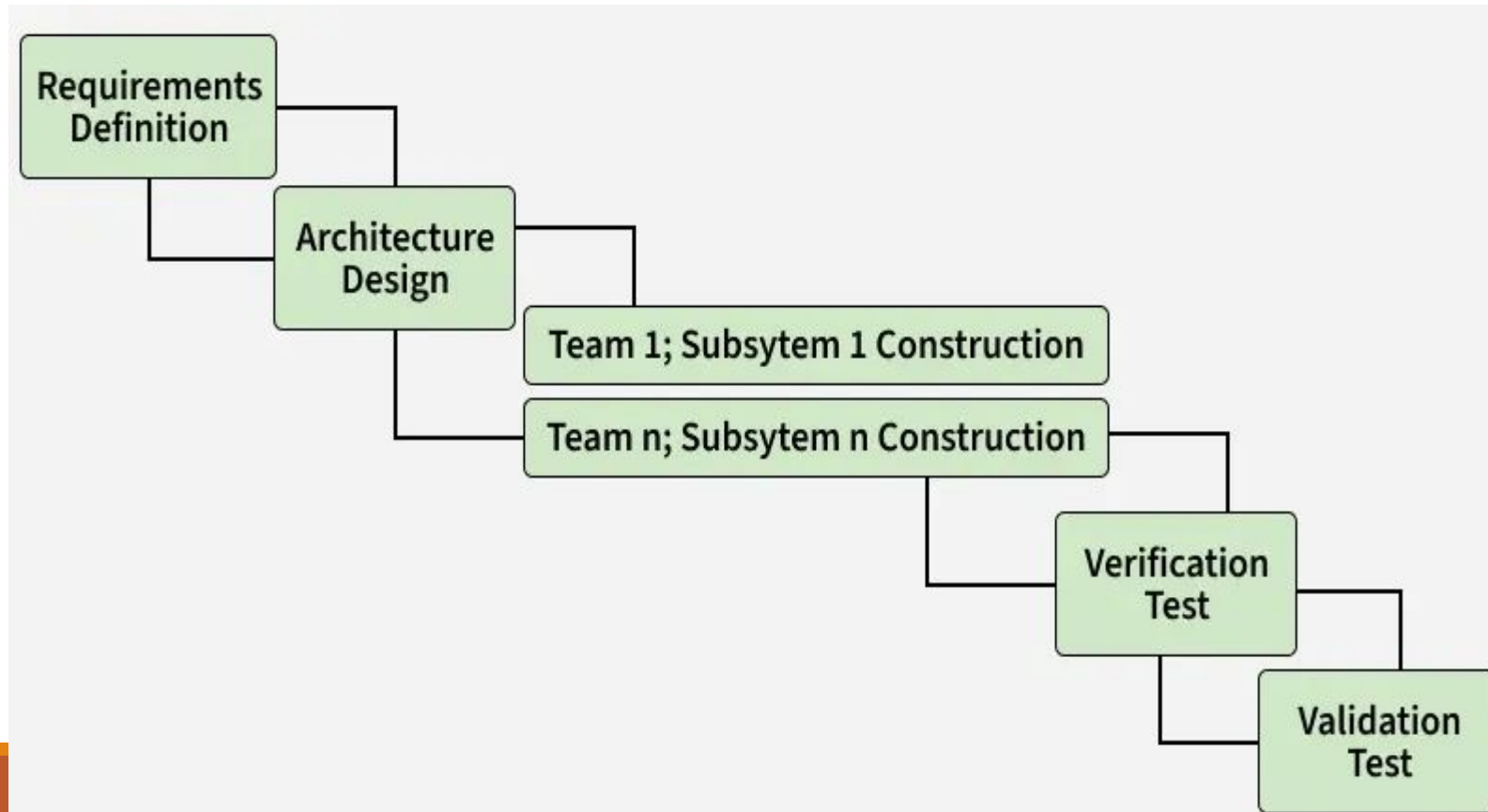
## Staged Delivery Model

- ❑ Develops software in a sequence of planned stages, where each stage delivers a functional part of the system. Each release brings the product closer to completion, allowing it to evolve gradually.
- ❑ Working versions are delivered at regular intervals, making progress visible and manageable throughout the development process. The diagram below shows this model :



## Parallel Development Model

- ❑ Divides the system into multiple modules that are developed simultaneously at the same time by different teams.
- ❑ By working on separate components in parallel, the development process becomes faster and more efficient. This approach reduces overall project time and allows teams to focus on specific functionalities concurrently. Given below is the diagram showing the model:



## Use Cases of Incremental Process Model

- **When the requirements are well-defined and clear**

Because increments can be planned and developed step-by-step with minimal requirement changes.

- **If the project has a long development timeline**

Incremental delivery helps manage complexity over time by breaking the project into smaller, manageable parts.

- **If the customer needs a quick product release**

You can deliver the most critical features early in the first increment, allowing the customer to start using the software sooner.

- **When you want to develop the most important features first**

This allows early feedback on key functionalities and better prioritization for subsequent increments.

## **Advantages**

- Early Software Delivery, improves customer satisfaction
- Easy to Manage Changes between increments
- Better Risk Management through early testing
- Cost Control by spreading budget over stages
- Simpler Debugging due to modular development
- High Software Quality through continuous testing

## **Disadvantages**

- Requires Skilled Team and proper planning
- Cost May Increase due to repeated testing and integration
- Design Issues if future requirements are unclear
- Integration Challenges between increments
- Repeated Bug Fixing across multiple increments

# Prototyping Model

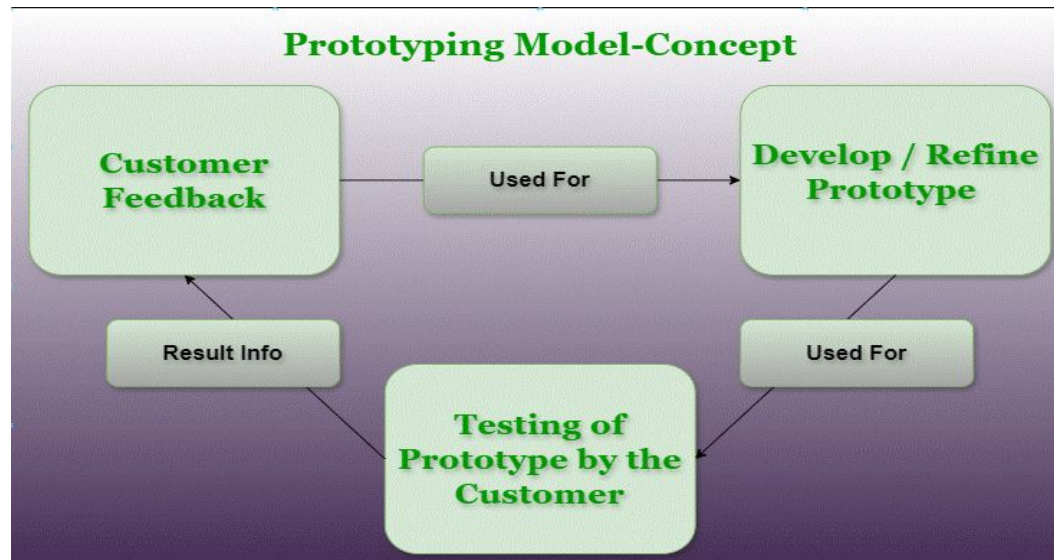
**Prototyping Model** is a way of developing software where an early version, or prototype, of the product is created and shared with users for feedback.

The Prototyping Model concept is described below:

## What is Prototyping Model?

The Prototyping Model is one of the most popularly used Software Development Life Cycle Models (SDLC models). This model is used when the customers do not know the exact project requirements beforehand.

In this model, a prototype of the end product is first developed, tested, and refined as per customer feedback repeatedly till a final acceptable prototype is achieved which forms the basis for developing the final product.

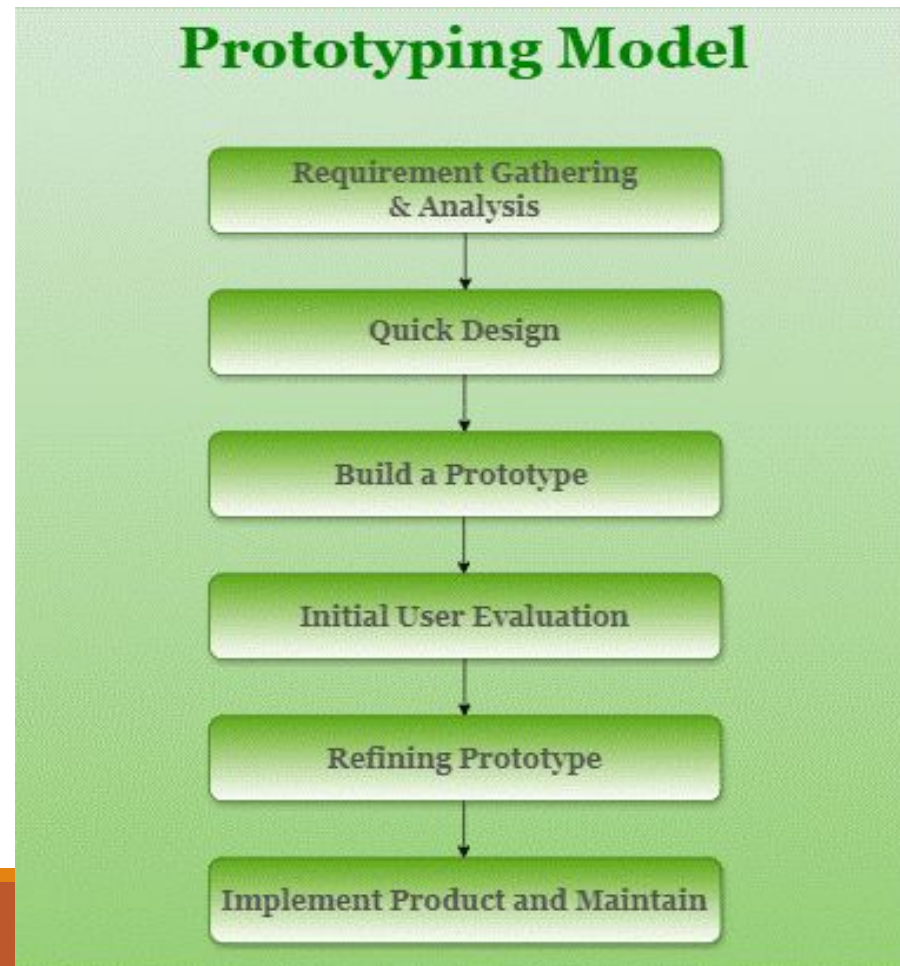




- ❑ In this process model, the system is partially implemented before or during the analysis phase thereby allowing the customers to see the product early in the life cycle. The process starts by interviewing the customers and developing the incomplete high-level paper model.
- ❑ This document is used to build the initial prototype supporting only the basic functionality as desired by the customer. Once the customer figures out the problems, the prototype is further refined to eliminate them. The process continues until the user approves the prototype and finds the working model to be satisfactory.

### Phases of Prototyping Model

Prototyping Model has six phases as follows:



## **1. Requirements gathering and analysis**

Requirement analysis is the first step in developing a prototyping model. During this phase, the system's desires are precisely defined. During the method, system users are interviewed to determine what they expect from the system.

## **2. Quick design**

The second phase could consist of a preliminary design or a quick design. During this stage, the system's basic design is formed. However, it is not a complete design. It provides the user with a quick overview of the system. The rapid design aids in the development of the prototype.

## **3. Build a Prototype**

During this stage, an actual prototype is intended to support the knowledge gained from quick design. It is a small low-level working model of the desired system.

#### **4. Initial user evaluation**

The proposed system is presented to the client for preliminary testing at this stage. It is beneficial to investigate the performance model's strengths and weaknesses. Customer feedback and suggestions are gathered and forwarded to the developer.

#### **5. Refining prototype**

If the user is dissatisfied with the current model, you may want to improve the type that responds to user feedback and suggestions. When the user is satisfied with the upgraded model, a final system based on the approved final type is created.

#### **6. Implement Product and Maintain**

The final system was fully tested and distributed to production after it was developed to support the original version. To reduce downtime and prevent major failures, the programmer is run on a regular basis.

## **Types of Prototyping Models**

There are four types of Prototyping Models, which are described below.

### **1. Rapid Throwaway Prototyping**

- This technique offers a useful method of exploring ideas and getting customer feedback for each of them.
- In this method, a developed prototype need not necessarily be a part of the accepted prototype.
- Customer feedback helps prevent unnecessary design faults and hence, the final prototype developed is of better quality.

### **2. Evolutionary Prototyping**

- In this method, the prototype developed initially is incrementally refined based on customer feedback till it finally gets accepted.
- In comparison to Rapid Throwaway Prototyping, it offers a better approach that saves time as well as effort.
- This is because developing a prototype from scratch for every iteration of the process can sometimes be very frustrating for the developers.

### 3. Incremental Prototyping

- In this type of incremental prototyping, the final expected product is broken into different small pieces of prototypes and developed individually.
- In the end, when all individual pieces are properly developed, then the different prototypes are collectively merged into a single final product in their predefined order.
- It's a very efficient approach that reduces the complexity of the development process, where the goal is divided into sub-parts and each sub-part is developed individually.
- The time interval between the project's beginning and final delivery is substantially reduced because all parts of the system are prototyped and tested simultaneously.
- Of course, there might be the possibility that the pieces just do not fit together due to some lack of preparedness in the development phase – this can only be fixed by careful and complete plotting of the entire system before prototyping starts.

## 4. Extreme Prototyping

This method is mainly used for web development. It consists of three sequential independent phases:

- In this phase, a basic prototype with all the existing static pages is presented in HTML format.
- In the 2nd phase, Functional screens are made with a simulated data process using a prototype services layer.
- This is the final step where all the services are implemented and associated with the final prototype.

This Extreme Prototyping method makes the project cycling and delivery robust and fast and keeps the entire developer team focused and centralized on product deliveries rather than discovering all possible needs and specifications and adding necessitated features.

## **Advantages of Prototyping Model**

- The customers get to see the partial product early in the life cycle. This ensures a greater level of customer satisfaction and comfort.
- New requirements can be easily accommodated as there is scope for refinement.
- Missing functionalities can be easily figured out.
- Errors can be detected much earlier thereby saving a lot of effort and cost, besides enhancing the quality of the software.
- The developed prototype can be reused by the developer for more complicated projects in the future.
- Flexibility in design.
- Early feedback from customers and stakeholders can help guide the development process and ensure that the final product meets their needs and expectations.

- Prototyping can be used to test and validate design decisions, allowing for adjustments to be made before significant resources are invested in development.
- Prototyping can help reduce the risk of project failure by identifying potential issues and addressing them early in the process.
- Prototyping can facilitate communication and collaboration among team members and stakeholders, improving overall project efficiency and effectiveness.
- Prototyping can help bridge the gap between technical and non-technical stakeholders by providing a tangible representation of the product.



## **Disadvantages of the Prototyping Model**

- Costly concerning time as well as money.
- There may be too much variation in requirements each time the prototype is evaluated by the customer.
- Poor Documentation due to continuously changing customer requirements.
- It is very difficult for developers to accommodate all the changes demanded by the customer.
- There is uncertainty in determining the number of iterations that would be required before the prototype is finally accepted by the customer.
- After seeing an early prototype, the customers sometimes demand the actual product to be delivered soon.
- Developers in a hurry to build prototypes may end up with sub-optimal solutions.
- The customer might lose interest in the product if he/she is not satisfied with the initial prototype.
- The prototype may not be scalable to meet the future needs of the customer.

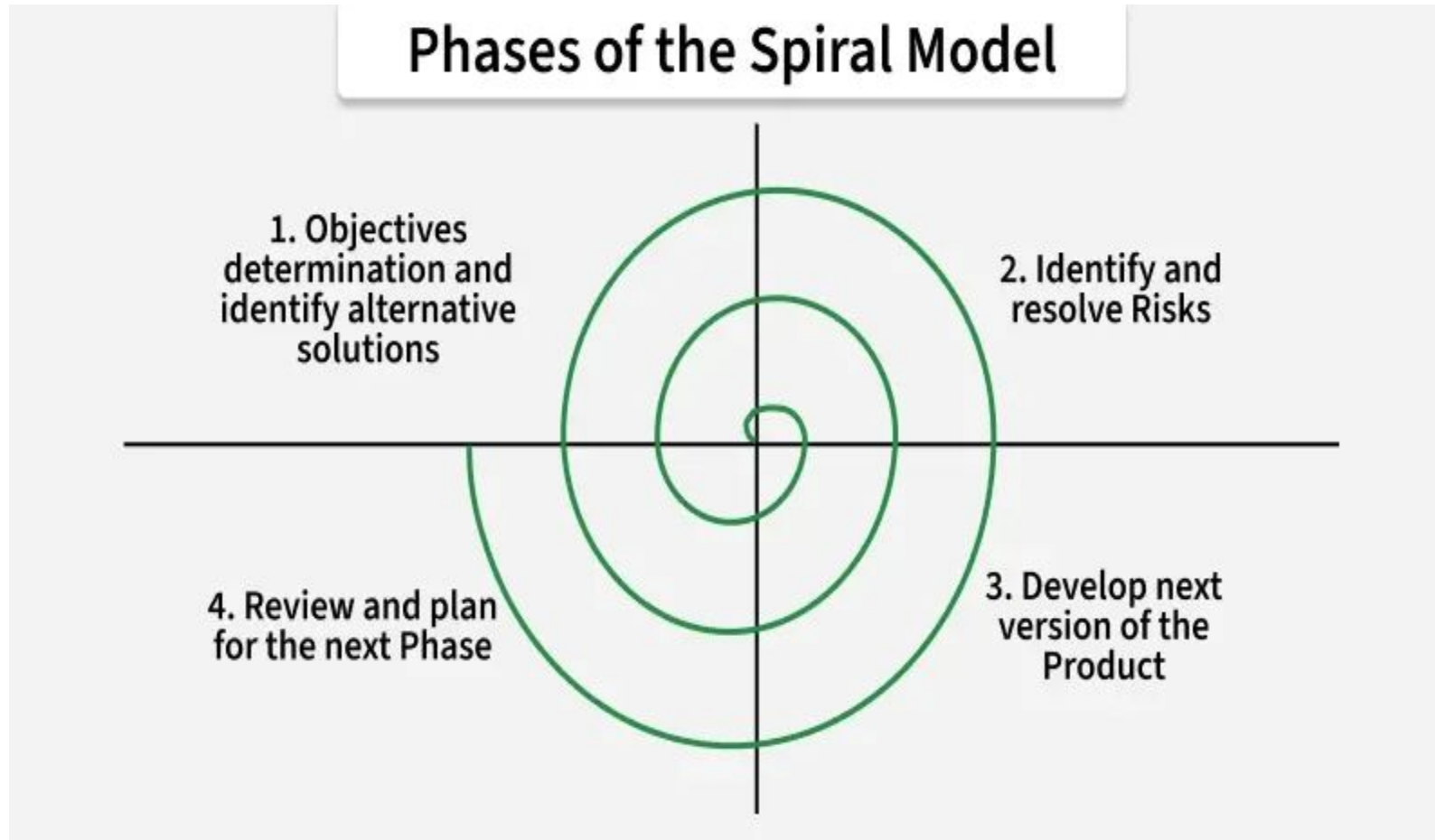
- The prototype may not accurately represent the final product due to limited functionality or incomplete features.
- The focus on prototype development may shift away from the final product, leading to delays in the development process.
- The prototype may give a false sense of completion, leading to the premature release of the product.
- The prototype may not consider technical feasibility and scalability issues that can arise during the final product development.
- The prototype may be developed using different tools and technologies, leading to additional training and maintenance costs.
- The prototype may not reflect the actual business requirements of the customer, leading to dissatisfaction with the final product.

## **Applications of Prototyping Model**

- The Prototyping Model should be used when the requirements of the product are not clearly understood or are unstable.
- The prototyping model can also be used if requirements are changing quickly.
- This model can be successfully used for developing user interfaces, high-technology software-intensive systems, and systems with complex algorithms and interfaces.
- The prototyping Model is also a very good choice to demonstrate the technical feasibility of the product.

# Spiral Model

The Spiral Model is one of the most important SDLC models, combining the **structured approach of the Waterfall Model** with the **flexibility of the Iterative Model**. It is mainly used for risk handling in large and complex projects . The Spiral Model was proposed by Barry Boehm.



# Spiral Model

The Spiral Model is a risk-driven and iterative software development model. Development progresses in a spiral shape consisting of multiple loops, where each loop represents a complete development cycle.

The number of loops depends on project size, complexity, and risk.

Each loop includes:

- Planning
- Risk analysis
- Development
- Evaluation

## Phases of the Spiral Model

Focus is on managing risk through multiple iterations of the software development process. Each phase of the Spiral Model is divided into four Quadrants.

## **1. Objectives Definition**

- Project goals and requirements are identified
- Functional and non-functional requirements are analyzed
- Possible solutions are explored

## **2. Risk Analysis and Resolution**

- Risks related to cost, schedule, performance, and technology are identified
- Best solution is selected
- Prototypes are built to reduce risks

## **3. Development and Testing**

- Selected features are designed, developed, and tested
- The next working version of the software is created

## 4. Review and Planning

- Customers evaluate the current version
- Feedback is collected
- Planning for the next iteration begins

The next iteration of the spiral begins with a new planning phase, based on the results of the evaluation.

## Risk Handling

- Risks are identified after development starts
- Prototypes help detect and resolve risks early
- Continuous risk evaluation at every iteration
- More flexible than Prototyping and Waterfall models

## Advantages

- **Effective Risk Handling:** Identifies and resolves risks at every phase, making it ideal for projects with high uncertainty.
- **Suitable for Large Projects:** Best suited for large and complex software systems.
- **Flexible Requirements:** Allows changes in requirements even at later stages of development.
- **Early Customer Feedback:** Customers can review early versions of the product, improving satisfaction.
- **Iterative Development:** Software is developed in repeated cycles, allowing gradual improvement.
- **Better Communication:** Regular reviews improve coordination between customers and developers.
- **High Software Quality:** Continuous testing and refinement lead to reliable and high-quality software.



## Disadvantages

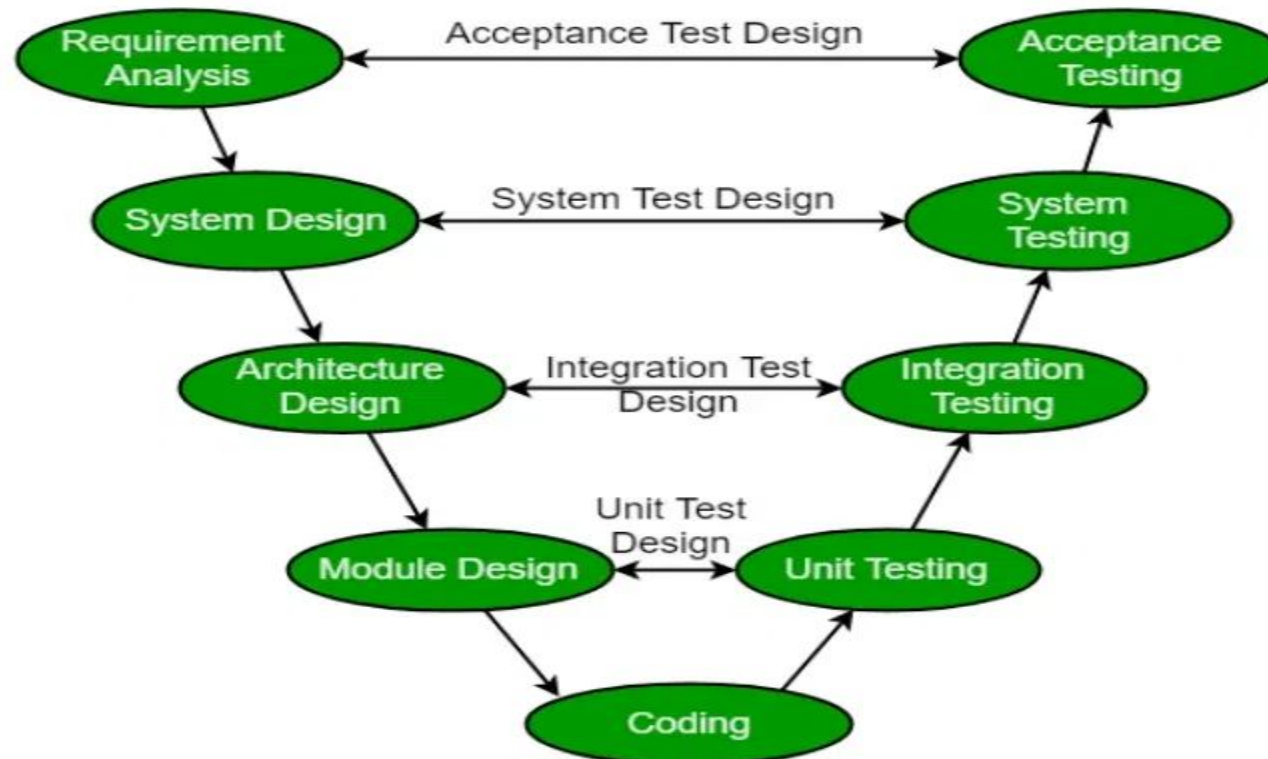
- **Complex to Manage:** The model is complex due to multiple iterations and continuous risk analysis.
- **High Cost:** Not suitable for small projects because it requires more time, effort, and resources.
- **Depends Heavily on Risk Analysis:** Success depends on accurate risk identification, which requires experienced professionals.
- **Difficult Time Estimation:** The number of iterations is not fixed, making scheduling difficult.
- **Time-Consuming:** Frequent reviews and evaluations can increase development time.
- **Resource Intensive:** Requires significant investment in planning, risk management, and testing.

# SDLC V-Model

The SDLC V-Model is a type of Software Development Life Cycle (SDLC). It is a method that includes testing and validation alongside each development phase. It creates a structure like the letter 'V,' which includes various phases that we will discuss in detail.

## Phases of SDLC V-Model

The V-Model, which includes the Verification and Validation it is a structural approach to software development. The following are the different Phases of the V-Model of the SDLC.



## **Importance of V-Model**

The V-Model is an important part of the SDLC, and the process is structured and sequential throughout all the testing. Here is why the V-Model is important:

- 1. Early Defect Identification**
- 2. Determining the Phases of Development and Testing**
- 3. Prevents "Big Bang" Testing**
- 4. Improves Cooperation**
- 5. Improved Quality Assurance**

## **Advantages of V-Model**

- This is a highly disciplined model and Phases are completed one at a time.
- V-Model is used for small projects where project requirements are clear.
- Simple and easy to understand and use.
- This model focuses on verification and validation activities early in the life cycle thereby enhancing the probability of building an error-free and good quality product.
- It enables project management to track progress accurately.
- The V-Model provides a clear and structured process for software development, making it easier to understand and follow.
- The V-Model places a strong emphasis on testing, which helps to ensure the quality and reliability of the software.
- The V-Model provides a clear link between the requirements and the final product, making it easier to trace and manage changes to the software.
- The clear structure of the V-Model helps to improve communication between the customer and the development team.

## **Disadvantages of the V-Model**

- The V-Model is a linear and sequential model, which can make it challenging to adapt to changing requirements or unforeseen events.
- The V-Model can be time-consuming, as it requires a lot of documentation and testing.
- High risk and uncertainty.
- It is not good for complex and object-oriented projects.
- It is not suitable for projects where requirements are not clear and contain a high risk of changing.
- This model does not support iteration of phases.
- The V-Model places a strong emphasis on documentation, which can lead to an overreliance on documentation at the expense of actual development work.

# Agile Methodology – Overview

**Agile methodology** is a modern software development approach that focuses on **flexibility, customer collaboration, and continuous improvement**. Instead of building the entire system at once, Agile develops software in **small, incremental cycles** called **iterations or sprints**.

## Core Characteristics of Agile

- **Iterative development** – Work is divided into small iterations (2–4 weeks) (Iterative means refining something in repeated cycles (iterations) to improve it, like adding features or fixing bugs. Incremental means building a product in small, functional parts (increments) that are added to the whole)
- **Incremental delivery** – Each iteration delivers a working feature
- **Customer involvement** – Continuous feedback from stakeholders
- **Adaptive planning** – Requirements can change anytime
- **Cross-functional teams** – Developers, testers, designers work together
- **Working software first** – Less documentation, more functionality

## **Agile Manifesto (4 Values)**

- 1. Individuals and interactions** over processes and tools
- 2. Working software** over comprehensive documentation
- 3. Customer collaboration** over contract negotiation
- 4. Responding to change** over following a plan

## **Agile Development Lifecycle**

- 1. Requirement Gathering** (User stories)
- 2. Planning** (Sprint planning)
- 3. Design**
- 4. Development**
- 5. Testing**
- 6. Deployment**
- 7. Review & Feedback**

## **Advantages of Agile**

- Faster delivery
- Better quality
- High customer satisfaction
- Easy to handle changing requirements
- Continuous improvement

## **Limitations of Agile**

- Less documentation
- Requires experienced team
- Difficult for very large or fixed-scope projects
- Hard to predict cost and timeline upfront



## Scrum Roles (Scrum Team)

In Scrum, there are **three core roles**. Together they form the **Scrum Team**.

### ① Product Owner (PO)

#### ◆ Role

Represents the **customer and business needs**.

#### ◆ Responsibilities

- Defines and manages the **Product Backlog**
- Writes and prioritizes **user stories**
- Clarifies requirements to the team
- Accepts or rejects completed work
- Ensures maximum **business value**

#### ◆ Key Skills

- Domain knowledge
- Decision-making
- Stakeholder communication

 *One Product Owner per Scrum Team*

## 2 Scrum Master (SM)

### ◆ Role

Acts as a **facilitator and coach** for the Scrum process.

### ◆ Responsibilities

- Ensures Scrum is understood and followed
- Conducts Scrum ceremonies (meetings)
- Removes **impediments** faced by the team
- Coaches team on Agile principles
- Acts as a bridge between PO and Development Team

### ◆ Key Skills

- Leadership
- Problem-solving
- Communication

 *Not a project manager, but a servant leader*

### 3 Development Team

#### ◆ Role

Builds the **actual product**.

#### ◆ Responsibilities

- Designs, develops, tests, and delivers increments
- Estimates effort for backlog items
- Self-organizes and manages its work
- Ensures quality of deliverables

#### ◆ Characteristics

- Cross-functional
- Self-organized
- Typically **3–9 members**
- No internal hierarchy

| Role             | Main Responsibility      |
|------------------|--------------------------|
| Product Owner    | What to build & priority |
| Scrum Master     | How to build (process)   |
| Development Team | Build the product        |

Scrum **does not include** roles like:

- Project Manager
- Team Lead
- Tester (separate role)



These responsibilities are **shared within the Scrum Team**.